

**REPORT ON
STUDENT INDUSTRIAL WORK EXPERIENCE SCHEME (SIWES)**

EXPERIENCES AND MINI PROJECT UNDERTAKEN AT

HiiT PLC

ON

**DESIGN AND IMPLEMENTATION OF A NETWORK-FLOW-BASED
TROJAN HORSE DETECTION SYSTEM FOR IoT DEVICES, WITH A
SIMULATED IP-CAMERA LABORATORY**

BY

[SURNAME, FIRST NAME, MIDDLE NAME]

[REGISTRATION NUMBER]

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING

UNIVERSITY OF UYO, UYO, AKWA IBOM STATE

COURSE CODE: [COURSE CODE]

SUBMITTED TO:

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING, UNIVERSITY OF UYO

**IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF A
BACHELOR OF SCIENCE (B.Sc.) DEGREE IN COMPUTER SCIENCE**

[MONTH, YEAR]

CERTIFICATION

This is to certify that this report on the Student Industrial Work Experience Scheme (SIWES) undertaken at HiiT Plc, together with the mini project on the design and implementation of a network-flow-based Trojan Horse detection system for IoT devices, was prepared by [SURNAME, FIRST NAME] (Reg. No. [REGISTRATION NUMBER]), a student of the Department of Computer Science, Faculty of Computing, University of Uyo, in partial fulfilment of the requirements for the award of the Bachelor of Science (B.Sc.) degree in Computer Science.

[Name of Industry-Based Supervisor]

Industry-Based Supervisor, HiiT Plc

Date: _____

[Name of Institution-Based Supervisor]

Institution-Based Supervisor, Department of Computer Science

Date: _____

[Name of SIWES Coordinator]

SIWES Coordinator, Department of Computer Science

Date: _____

[Name of Head of Department]

Head, Department of Computer Science

Date: _____

ACKNOWLEDGEMENTS

I thank Almighty God for the health, strength and understanding that carried me through my industrial training and the preparation of this report.

I am grateful to the management and staff of HiiT Plc for admitting me into the organisation and for giving me room to learn by doing. Special thanks go to my industry-based supervisor, [name of supervisor], and to the facilitators and colleagues who reviewed my work, answered my questions and challenged me to explain my design decisions.

I appreciate the Head of the Department of Computer Science, the SIWES Coordinator and my institution-based supervisor, [name], at the University of Uyo for their supervision, advice and administrative support, and the Industrial Training Fund (ITF) for sustaining a scheme that lets students meet industry before they graduate.

Finally, I thank my family, friends and course mates for their prayers, encouragement and patience throughout the placement.

ABSTRACT

This report presents my Student Industrial Work Experience Scheme (SIWES) training at HiiT Plc, a Nigerian information technology company engaged in IT training, publishing, consultancy and software solutions. Over the placement I worked mainly on one engineering problem: how to detect a Trojan Horse on Internet-of-Things (IoT) devices such as IP cameras when the devices themselves cannot run security software.

The mini project, presented in Chapter Four, is a network-flow-based detection system. It accepts flow records as a CSV file (timestamp, device, source and destination address, destination port, protocol and bytes sent), compares them with an optional baseline of known-good behaviour, and reports nine kinds of findings, among them contact with known-bad addresses, suspicious ports, periodic beaconing, scan bursts and large outbound transfers. Each finding is mapped to a MITRE ATT&CK technique, each device receives a risk score and a status of clean, suspicious or compromised, and a flagged device can be quarantined from a web interface. To demonstrate the system safely I built a Docker laboratory with simulated IP cameras, a looping video stream, a fake command-and-control server and a harmless trojan simulator on an isolated network.

The system was implemented in Python and Flask and verified with 15 automated tests together with positive, negative and laboratory tests. On the bundled infected sample it reported 22 findings on one camera, marked it compromised, and left the other two cameras and the clean capture free of findings. The report also covers the experience gained in programming, networking, cybersecurity, containers, testing, version control and technical documentation, assesses the host organisation, and offers recommendations to the company, the University and the ITF.

Keywords: SIWES, HiiT Plc, IoT security, Trojan Horse, network flow, beaconing, MITRE ATT&CK, Python, Flask, Docker.

TABLE OF CONTENTS

CERTIFICATION.....	5
ACKNOWLEDGEMENTS.....	5
ABSTRACT.....	5
CHAPTER ONE: INTRODUCTION: BRIEF HISTORY OF SIWES.....	7
1.1 Introduction.....	7
1.2 Brief History of SIWES in Nigeria.....	7
1.3 Stakeholders and Their Roles.....	7
1.4 Aims and Objectives of SIWES.....	8
1.5 Relevance of SIWES to Computer Science Students.....	8
1.6 Purpose and Structure of This Report.....	8
CHAPTER TWO: BRIEF HISTORY OF THE SIWES ORGANISATION (HiiT PLC) 9	
2.1 Introduction.....	9
2.2 Background and History of HiiT Plc.....	9
2.3 Vision, Mission and Core Values.....	9
2.4 Services and Business Units.....	10
2.5 Organisational Units and My Interaction with Them.....	10
2.6 Placement Details.....	11
2.7 Projects I Worked On During the Training Period.....	11
2.8 Review of HiiT Plc.....	12
CHAPTER THREE: EXPERIENCE GAINED AND WORK DONE.....14	
3.0 Introduction to the Training Environment.....	14
3.1 Overview of Experiences Gained.....	14
3.2 Network Flow Analysis.....	14
3.3 Cybersecurity Concepts and Ethical Practice.....	15
3.4 Containers and the Virtual Laboratory.....	15
3.5 Programming with Python and Flask.....	16
3.6 Software Testing and Debugging.....	16
3.7 Version Control and Collaboration.....	16
3.8 Technical Documentation and Professional Communication.....	17
3.9 Soft Skills and Professional Conduct.....	17
3.10 Challenges Encountered and How They Were Solved.....	17
3.11 Conclusion.....	18
CHAPTER FOUR: MINI PROJECT: DESIGN AND IMPLEMENTATION OF A NETWORK-FLOW-BASED TROJAN HORSE DETECTION SYSTEM FOR IoT DEVICES.....	19
4.0 Introduction.....	19
4.1 Background.....	19

4.2 Problem Statement.....	19
4.3 Aim and Objectives.....	19
4.4 Scope and Limitations.....	20
4.5 Tools and Technologies.....	20
4.6 Methodology.....	20
4.7 System Requirements.....	21
4.8 System Design.....	21
4.9 Implementation.....	24
4.10 Testing and Results.....	29
4.11 Discussion of Results.....	32
4.12 Challenges Encountered.....	32
4.13 Limitations.....	32
4.14 Recommendations and Future Work.....	33
4.15 Conclusion.....	33
CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS.....	34
5.1 Summary.....	34
5.2 Knowledge and Skills Acquired.....	34
5.3 Conclusion.....	34
5.4 Recommendations.....	34
REFERENCES.....	36
APPENDIX A: CORE SOURCE CODE OF THE DETECTION SYSTEM.....	37
APPENDIX B: SIWES LOGBOOK EXTRACT.....	40
APPENDIX C: CERTIFICATES AND SUPPORTING DOCUMENTS.....	41
APPENDIX D: GLOSSARY OF TECHNICAL ACRONYMS.....	42

CHAPTER ONE: INTRODUCTION: BRIEF HISTORY OF SIWES

1.1 Introduction

The Student Industrial Work Experience Scheme (SIWES) is a practical training programme that forms part of the approved curriculum for degrees, diplomas and certificates in Nigerian universities, polytechnics and colleges of education. Students of Computer Science, engineering, the sciences, agriculture, medicine and similar disciplines are required to complete it before they can graduate. The thinking behind it is straightforward: a graduate should already have seen how a real organisation works before the first day of formal employment.

During SIWES a student spends a defined period, normally several months, away from lectures and in an organisation whose activities match the field of study. The student works under supervision, uses professional tools, follows workplace rules and records the activities in a logbook. In return the student gains skills and habits that are hard to acquire in a classroom, such as punctuality, accountability, teamwork, documentation and delivering to a deadline.

1.2 Brief History of SIWES in Nigeria

The Industrial Training Fund (ITF) was established in 1971, and in 1973 it launched SIWES in response to a recurring complaint from employers: Nigerian graduates, particularly in science and technology, understood theory well but lacked practical competence, so companies had to spend heavily on retraining them. The scheme was designed to close that gap by making industrial exposure part of the degree itself.

The first cohorts were small and the ITF ran the scheme on its own. In the years that followed, coordination moved between the ITF and the academic regulators, and the Federal Government later restored operational responsibility to the ITF while the regulators kept control of academic standards. SIWES has since grown from a handful of institutions to a nationwide programme covering a wide range of disciplines.

1.3 Stakeholders and Their Roles

SIWES depends on cooperation between several parties. Table 1.1 summarises who does what.

Table 1.1: Stakeholders in SIWES and Their Roles

Stakeholder	Role
Industrial Training Fund (ITF)	Administers the scheme, supports placement, supervises students and contributes to their funding and allowances.
NUC, NBTE and NCCE	Set and monitor academic standards for universities, polytechnics and colleges of education respectively, including the SIWES component of programmes.
Tertiary institution	Registers students, helps with placement, appoints supervisors, assesses the report and logbook, and awards credit.
Host organisation	Provides placement, equipment, mentoring and a practical assessment of the student.

Stakeholder	Role
Student	Attends the placement diligently, keeps the logbook, completes assigned tasks and submits a report.

1.4 Aims and Objectives of SIWES

The main aim of SIWES is to prepare students for work by giving them supervised, practical exposure in their field of study. Its specific objectives are:

- To let students apply classroom knowledge to real problems in real working environments.
- To develop the practical and professional skills that employers expect of graduates.
- To teach how organisations are structured and how work flows through them.
- To build discipline, integrity, responsibility and good work ethics.
- To improve communication, teamwork and leadership ability.
- To improve graduate employability and reduce the cost of post-graduation retraining.
- To strengthen links between institutions and industry.

1.5 Relevance of SIWES to Computer Science Students

Computing changes faster than any syllabus can follow. Virtualisation, containers, cloud platforms, security tooling and version control are best learned by using them, and a placement provides the chance to see how software is specified, built, tested, secured, documented and delivered by a team. For me, SIWES was also an opportunity to test an interest in cybersecurity: instead of reading about attacks in the abstract, I built a laboratory in which an attack could be simulated, detected and contained.

1.6 Purpose and Structure of This Report

This report documents what I did and learned at HiiT Plc. Chapter One introduces SIWES. Chapter Two describes HiiT Plc, its structure and services, and the work I carried out there. Chapter Three records the experience gained. Chapter Four presents the mini project in full, from requirements and design to implementation, testing and results. Chapter Five summarises the training, draws conclusions and makes recommendations. The references and appendices follow.

CHAPTER TWO: BRIEF HISTORY OF THE SIWES ORGANISATION (HiiT PLC)

2.1 Introduction

HiiT Plc is a Nigerian-owned information technology company registered with the Computer Professionals Registration Council of Nigeria (CPN). It operates in four connected areas: IT training and education, publishing, IT consultancy, and IT solutions development and services. This chapter describes the organisation, the units I interacted with, the projects I worked on and my assessment of the company.

2.2 Background and History of HiiT Plc

HiiT Plc grew out of the belief that Nigerians should be able to acquire world-class IT skills from Nigerian professionals, at international standards but at a cost and in a form that suit local conditions. Public information describes it as the largest indigenous IT training company in Nigeria, with CPN-accredited training centres in Lagos, Abuja, Ibadan and Kano, and more than 60,000 graduates over about a quarter of a century of operation (BusinessDay, 2020). The company was founded in 1996 and has its head office in Lagos [confirm founding year and head-office address from the company profile].

Beyond its physical centres, HiiT delivers instructor-led online training so that learners elsewhere in Nigeria can attend the same classes, and it runs a SIWES and internship programme through which students from tertiary institutions spend time working with its facilitators and practitioners. My placement took place within that programme.

2.3 Vision, Mission and Core Values

[Replace this paragraph with the company's official vision and mission statements from the orientation handbook or website.] In summary, HiiT aims to be a leading indigenous centre of excellence that produces skilled, certified and employable IT professionals, and to deliver quality IT education, consultancy and software services through experienced facilitators, current course content and good facilities.

The values the company works by, as presented in its profile, are:

- **Excellence:** treating the minimum acceptable standard as a starting point, not a target.
- **Innovation:** continuously looking for better methods, tools and opportunities as technology changes.
- **Caring:** showing concern for learners, clients and colleagues beyond the transaction.
- **Happiness:** building a working and learning environment in which people can thrive.

2.4 Services and Business Units

Table 2.1: Lines of Business of HiiT Plc

Line of business	Description
IT training and education	Instructor-led and online courses in programming, networking, databases, cybersecurity, digital literacy and certification preparation.
Software development and services	Design and delivery of software solutions and IT services tailored to client needs.
IT consultancy	Advice to organisations on systems, infrastructure, ICT policy and digital adoption.
Networking, ICT and CBT centre set-up	Planning, equipping and commissioning networks, ICT laboratories and computer-based testing centres.
Publishing	Production of IT textbooks and learning materials for training and self-study.
SIWES and internship programme	Supervised industrial training with mentoring, practical tasks and exposure sessions for students.

2.5 Organisational Units and My Interaction with Them

At the start of the placement I was shown how the organisation is arranged. Table 2.2 lists the units I dealt with and how each related to my work.

Table 2.2: Units of HiiT Plc and Their Relevance to My Work

Unit	Function	Relevance to my work
Training / academic unit	Course content, facilitators, classes and certification.	Facilitators reviewed my approach and explained concepts in networking and security.
Software and solutions unit	Development, testing and deployment of client applications.	Source of the working practices I followed: modular code, tests and version control.
Networking and infrastructure unit	Laboratories, networks, hardware and technical support.	Provided the equipment and network access my laboratory work relied on.
Publishing and content unit	Preparation and editing of learning materials.	Showed the standard expected of written technical material.
SIWES / internship desk	Placement, attendance, supervision and intern activities.	Assigned my supervisor and tracked my attendance and logbook.

Seeing these units working side by side taught me that technical work is never done in isolation: a single deliverable passes through requirements, development, review, documentation and support before it becomes useful to anyone.

2.6 Placement Details

Table 2.3: Summary of My Placement

Item	Details
Host organisation	HiiT Plc
Branch / centre	[branch or centre]
Unit / department	[unit or department]
Period of training	[start date] to [end date]
Industry-based supervisor	[name and designation]
Institution-based supervisor	[name and designation]
Main assignment	Design and implementation of a network-flow-based Trojan Horse detection system for IoT devices, with a simulated camera laboratory

2.7 Projects I Worked On During the Training Period

Table 2.4 lists the technical work I completed. The detection system is the mini project and is presented fully in Chapter Four; the supporting activities are described in Chapter Three.

Table 2.4: Projects and Work Packages Completed

S/N	Work package	Domain	Main tools	Outcome
1	Flow-analysis and detection engine	Cybersecurity / software	Python (standard library)	Parser, baseline builder, nine detection rules and risk scoring
2	Web interface and JSON API	Software engineering	Flask, HTML, JavaScript	CSV upload, results display, quarantine and release
3	Simulated IP-camera laboratory	Systems / networking	Docker Compose, MediaMTX, ffmpeg	Isolated lab with cameras, RTSP stream and shared telemetry
4	Harmless trojan and fake C2 simulator	Security	Python sockets	Repeatable infection, beaconing and containment demonstration
5	Testing and verification	Quality assurance	pytest	15 automated tests plus positive, negative and laboratory tests
6	Documentation and screenshots	Technical writing	Playwright, Chromium, Word	README, PDF guide, Word report and annotated screenshots
7	[other task assigned at HiiT, if any]	[domain]	[tools]	[outcome]

2.8 Review of HiiT Plc

2.8.1 Strengths

- **Longevity and credibility:** A quarter of a century in a sector where many start-ups disappear is a strong record.
- **Professional standing:** CPN registration reassures learners, interns and clients that the company works within professional standards.
- **Breadth of services:** Training, publishing, consultancy and software under one roof lets an intern see several sides of the industry.
- **National reach:** Centres in several cities, together with online delivery, make the training accessible beyond Lagos.
- **Willing mentors:** Facilitators explained the reasoning behind their advice instead of only giving instructions.

2.8.2 Areas for Improvement

- A written schedule of tasks and learning outcomes at the start of the placement would make expectations visible from the first week.
- More project-based assignments with a named reviewer would convert exposure into measurable skill.
- Reliable backup power and redundant internet access would protect laboratory hours from the infrastructure problems that affect most Nigerian organisations.
- Supervised portfolio presentations would help interns show completed work to future employers.

2.8.3 SWOT Summary

Table 2.5: SWOT Analysis of HiiT Plc

Strengths	Weaknesses
Long operating history	Intern task scheduling could be more formal
CPN-registered, Nigerian-owned	Resource limits at some centres
Several service lines	Dependence on unreliable national infrastructure
Multi-city presence plus online delivery	

Opportunities	Threats
Growing demand for cybersecurity, data and software skills	Competition from global online platforms
Growth of remote and hybrid learning	Rapid change in technology and certifications
Partnerships with universities for SIWES	Cost pressures on students

2.8.4 My Overall Assessment

My assessment of HiiT Plc is positive. The company takes the training of Nigerians seriously, and that showed in the way facilitators tied every concept to real use. I would recommend it to Computer Science students who want practical exposure to programming, networking and security. [Adjust this paragraph to reflect your own experience.]

CHAPTER THREE: EXPERIENCE GAINED AND WORK DONE

3.0 Introduction to the Training Environment

My placement at HiiT Plc ran from [start date] to [end date] at [branch or centre] under the supervision of [supervisor name]. [Describe orientation: working hours, conduct and safety rules, introduction to the units, and the tools provided.] I was then given progressively more demanding tasks, beginning with environment set-up and ending with independent project work.

My development platform was [describe your laptop or workstation: model, processor, memory and operating system]. The main software used was Python 3, the Flask web framework, Docker and Docker Compose, the pytest test framework, Git and GitHub, a code editor, and Microsoft Word for documentation. Working within the limits of the machine taught me to plan resources carefully, which is a professional habit in its own right.

3.1 Overview of Experiences Gained

The training strengthened my competence in six areas:

- (1) **Networking and traffic analysis:** how conversations between devices are summarised as flows, and what normal and abnormal flows look like.
- (2) **Cybersecurity of IoT devices:** how trojans persist, communicate and spread, how defenders describe those behaviours, and how to study them safely.
- (3) **Systems and containers:** building an isolated, repeatable laboratory with Docker Compose.
- (4) **Software development in Python:** structuring a program into a testable engine, a web layer and simulators.
- (5) **Quality assurance and version control:** writing automated tests, finding defects and managing changes in Git.
- (6) **Professional practice:** technical documentation, reporting, communication and time management.

3.2 Network Flow Analysis

Most of my early learning was about the data the detector works on. A flow record summarises one conversation between two endpoints: when it started, the addresses and ports involved, the protocol and how many bytes moved. Flow data is exported by routers and switches (NetFlow and IPFIX) and by monitoring tools such as Zeek and Suricata. It is attractive for IoT monitoring because it is compact, it does not need access to the device and it works even when the payload is encrypted.

I learned to read flows for behaviour rather than content. A camera that streams video to a recorder every ten seconds, synchronises its clock at fixed times and sends a periodic heartbeat to its vendor has a clear, repetitive pattern. A trojan leaves a different pattern: contact with an

address the device never used, at a constant interval, on an unusual port, sometimes followed by a burst of connections to neighbouring hosts. Distinguishing the two is the central problem the project solves.

3.3 Cybersecurity Concepts and Ethical Practice

3.3.1 Threat Background

IoT cameras typically run a small Linux system with a web administration page and a video service, and many ship with default credentials. The Mirai botnet of 2016 showed how quickly such devices can be taken over when those defaults are left in place (Antonakakis et al., 2017). An infected device usually waits for instructions from a command-and-control (C2) server, scans for more victims and takes part in attacks, and each of these produces network traffic that differs from the device's ordinary role.

3.3.2 MITRE ATT&CK

I learned to describe detections using the MITRE ATT&CK knowledge base, which gives each adversary technique an identifier, for example T1071 (Application Layer Protocol), T1571 (Non-Standard Port), T1046 (Network Service Discovery) and T1041 (Exfiltration Over C2 Channel). Using these identifiers in the detector means findings can be understood by any security team without further explanation.

3.3.3 Ethical and Legal Framework

Security work carries responsibilities. I worked only on systems I had built myself, inside an isolated laboratory, and I deliberately avoided real malware: the trojan in the lab is a simulator that reproduces observable indicators and nothing else. I also became aware that unauthorised access to computer systems is an offence under Nigerian law, including the Cybercrimes (Prohibition, Prevention, etc.) Act 2015, which is a good reason to keep testing confined to environments one owns or is authorised to test.

3.4 Containers and the Virtual Laboratory

To demonstrate the detector without real malware or real cameras, I built a laboratory with Docker Compose. It defines simulated cameras, a looping video stream, a fake C2 server, the trojan simulator and the detector, all connected to a network declared as internal so that nothing can reach the internet. A separate network exposes only the web interface, one camera admin page and the video port to the host. A shared volume carries the flow records from the simulators to the detector. Figure 3.1 shows the layout.

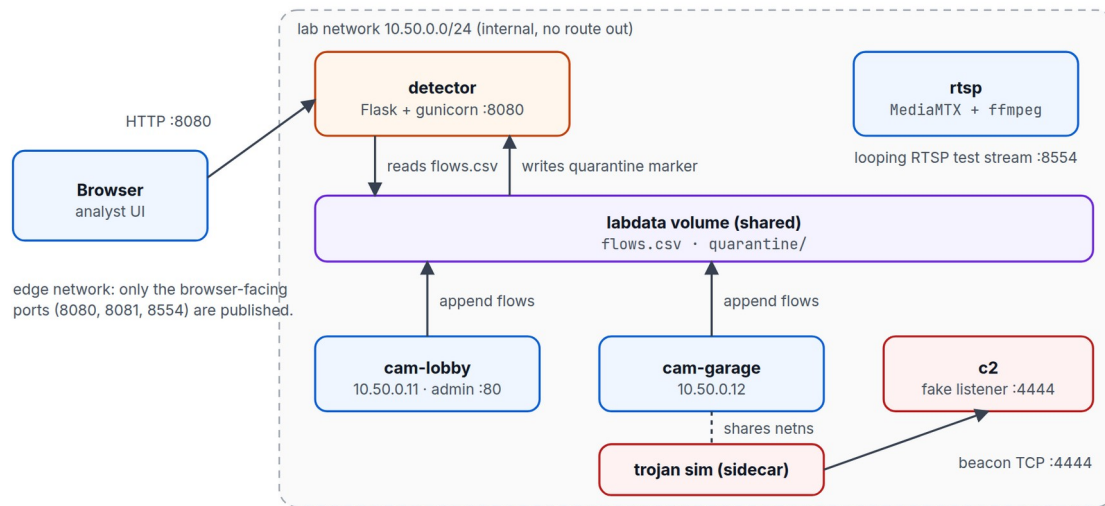


Figure 3.1: Architecture of the laboratory and the detector

Working with containers taught me how images, networks, volumes and service profiles fit together, and why an infection step should be an explicit action: the trojan service sits behind a Compose profile, so the laboratory starts clean and is infected only when I choose.

3.5 Programming with Python and Flask

The software is organised in layers so that each part can be understood and tested on its own. The analysis engine uses only the Python standard library and has no dependency on the web framework. The Flask application is a thin layer that receives uploads, calls the engine and returns JSON. The simulators are small separate programs. Keeping the engine independent made testing straightforward and would allow it to be reused in a different front end.

I practised several habits that I had only met in theory before: validating input and returning clear error messages instead of stack traces; checking file names before using them; keeping thresholds as named constants so they can be tuned; and returning structured data (findings, device summaries) rather than formatted text.

3.6 Software Testing and Debugging

I wrote 15 automated tests with pytest covering the parser, the detection rules, the web endpoints and the failure cases. Testing was valuable mostly because it exposed mistakes I would not have noticed by inspection. Two defects in particular changed the design: legitimate periodic traffic was at first reported as beaconing, and the lab's stand-in internet addresses were wrongly treated as private, which silently disabled the large-transfer rule. Both are explained in Section 4.10.2. I learned that a failing test is information about the design and not just an obstacle.

3.7 Version Control and Collaboration

The project was developed in a Git repository hosted on GitHub (emmanuelekopimo/build-with-ai) on a dedicated feature branch, with small commits that each describe one change, and a

pull request used to present the work for review. I learned why history matters: a clear commit message makes a change understandable weeks later, and a branch keeps unfinished work away from the stable code.

3.8 Technical Documentation and Professional Communication

Documentation formed a large part of the work. I wrote a README with a quick start and a table of commands, and automated the documentation itself: a script drives the application in a headless browser, takes screenshots and draws numbered callouts on key elements, and further scripts assemble a PDF guide and a Word report from the same source so that the text and the numbers cannot drift apart. I also prepared a timed five-minute presentation script for demonstrating the system. Writing for different readers, a developer, a supervisor and an audience, showed me how much the same facts need to be reshaped for each.

3.9 Soft Skills and Professional Conduct

- **Punctuality and discipline:** Keeping to working hours and delivering tasks on the agreed days.
- **Communication:** Explaining technical decisions to supervisors and facilitators in plain language, and asking for clarification early.
- **Receiving feedback:** Treating review comments as input to improve the work.
- **Time management:** Breaking the project into stages and finishing each one before starting the next.
- **Integrity:** Staying within the ethical boundaries of security work and reporting limitations honestly.

[Add or adjust points to reflect your own experience at HiiT.]

3.10 Challenges Encountered and How They Were Solved

Table 3.1: Challenges and Solutions

Challenge	How it was solved
Healthy cameras were flagged because their regular traffic looked like beaconing.	Beaconing is now judged against a per-device baseline, so regular traffic to known peers is ignored while the same regularity to a new peer is reported.
The large-transfer rule did not fire on the simulated upload.	The standard library treats documentation address ranges as private. I replaced it with an explicit list of LAN ranges.
No Docker daemon was available in the development environment.	I ran the simulators, the fake C2 and the detector directly as Python processes to verify the laboratory logic, and documented that the container layer still needs a run on a machine with Docker.
Screenshots of long tables spilled over several pages of the documentation.	The capture script now shows the first rows and a "more findings not shown" line.

Challenge	How it was solved
The baseline is kept in memory and may not be shared between server workers.	Recorded as a known limitation, with single-worker or shared-storage options recommended.
[Add a challenge you faced at HiiT]	[How you solved it]

3.11 Conclusion

The placement gave me practical experience in network analysis, cybersecurity, containers, Python development, testing, version control and documentation. The most important lesson was that good engineering is iterative: design, build, test, find what is wrong, correct it and write down what was learned.

CHAPTER FOUR: MINI PROJECT: DESIGN AND IMPLEMENTATION OF A NETWORK-FLOW-BASED TROJAN HORSE DETECTION SYSTEM FOR IoT DEVICES

4.0 Introduction

This chapter presents the mini project I carried out at HiiT Plc. It follows the usual structure of a software engineering project: background, problem, aim and objectives, tools, methodology, requirements, design, implementation, testing and results, discussion, limitations and conclusions.

4.1 Background

IoT devices such as IP cameras are inexpensive, always connected and often protected only by default passwords. Once compromised, a camera continues to stream video while it quietly contacts a command-and-control server, scans other hosts or uploads data. Because the device cannot host security software, the behaviour has to be detected from outside, in the traffic it produces. Flow records, which summarise who talked to whom, on which port, how often and how much, provide exactly that view without needing the payload.

4.2 Problem Statement

- **No on-device protection:** IoT devices cannot run endpoint security agents, so detection must happen on the network.
- **Signature dependence:** Signatures recognise only malware that has already been analysed; behaviour is a more durable indicator.
- **Legitimate regularity:** Healthy camera traffic is also periodic, so a naive beacon detector produces false positives.
- **Safe evaluation:** Detection logic must be demonstrated without handling real malware and without risk to real networks.

4.3 Aim and Objectives

Aim: to design, implement and test a lightweight, agentless system that detects trojan-like behaviour on IoT devices from network-flow data, explains its findings in standard terms and supports containment, together with a safe laboratory for demonstrating it.

Objectives:

- (1) Define a tool-neutral flow-record CSV schema that can be produced from Zeek, Suricata or router exports.
- (2) Implement a detection engine for known-bad destinations, suspicious and unusual ports, beaconing, scan bursts and large outbound transfers.
- (3) Support an optional per-device baseline so that legitimate periodic traffic is not reported.

- (4) Map every finding to a MITRE ATT&CK technique and compute a per-device risk score and status.
- (5) Provide a web interface and API for uploading CSV files, viewing findings and quarantining a device.
- (6) Build an isolated Docker laboratory with simulated cameras, a harmless trojan and a fake C2 server.
- (7) Verify the system with automated, positive, negative and laboratory tests.

4.4 Scope and Limitations

The system analyses flow records; it does not inspect payloads, use machine learning or see inside encrypted traffic. Its rules are heuristic and were tuned on simulated data. Flow telemetry in the laboratory is produced by the simulators rather than captured from a wire, quarantine in the laboratory is a marker file, and the baseline is held in memory. These limits are discussed in Section 4.13.

4.5 Tools and Technologies

Table 4.1: Tools and Technologies Used

Tool	Purpose
Python 3.11+	Analysis engine, web application and simulators.
Flask and gunicorn	Web interface, JSON API and production server.
Docker and Docker Compose	Isolated multi-container laboratory with internal and edge networks.
MediaMTX and ffmpeg	Looping synthetic RTSP video stream for the simulated cameras.
pytest	Automated testing.
Git and GitHub	Version control and pull-request review.
Playwright and Chromium	Automated screenshots and PDF generation for the documentation.
Microsoft Word	Editable project report.

4.6 Methodology

I used an incremental approach in which each stage produced something that could be run and tested before the next began. The stages were: (1) define the flow schema and generate deterministic sample data; (2) write the parser and a first set of rules and test them on the samples; (3) add the baseline and refine the rules after the defects found in testing; (4) add the web application and API; (5) build the simulators and the Docker laboratory; (6) add quarantine and release; (7) write the documentation and automate its screenshots. Writing the tests alongside each stage made it safe to change the rules later.

4.7 System Requirements

4.7.1 Functional Requirements

Table 4.2: Functional Requirements

ID	Requirement
FR1	The system shall accept a flow CSV upload and optionally a baseline CSV.
FR2	The system shall accept common column aliases and ISO-8601 or epoch timestamps.
FR3	The system shall reject malformed or incomplete files with a message naming the problem.
FR4	The system shall detect contact with known-bad addresses and use of suspicious or unusual ports.
FR5	The system shall detect periodic beaconing to peers outside a device's baseline.
FR6	The system shall detect scan-like bursts and large outbound transfers.
FR7	The system shall give every finding a severity, evidence and a MITRE ATT&CK technique.
FR8	The system shall compute a per-device risk score and a status of clean, suspicious or compromised.
FR9	The user shall be able to quarantine and release a device.
FR10	The system shall provide a laboratory that can infect and clean a simulated camera safely.

4.7.2 Non-Functional Requirements

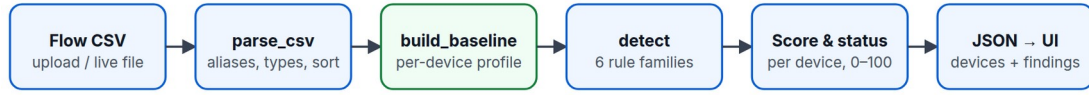
Table 4.3: Non-Functional Requirements

ID	Requirement
NFR1	Safety: no real malware; the laboratory network has no route to the internet.
NFR2	Repeatability: sample data and laboratory behaviour are deterministic and can be regenerated.
NFR3	Usability: the interface is a single page, readable on a phone, with clear error messages.
NFR4	Maintainability: the engine is independent of the web layer and thresholds are named constants.
NFR5	Security of the application: uploads are size-limited, output is escaped and device names are validated.
NFR6	Portability: runs with Python alone or with Docker Compose.

4.8 System Design

4.8.1 Architecture and Processing Pipeline

The overall architecture is the one shown in Figure 3.1. Inside the detector, data moves through the pipeline in Figure 4.1: the CSV is parsed and normalised, an optional baseline is built from known-good traffic, six rule families are evaluated for each device, findings are scored, and a JSON result is returned to the interface.



baseline is optional: from a second upload, or frozen from the live capture

Figure 4.1: Detection pipeline from flow CSV to scored result

4.8.2 Data Model

Table 4.4: Flow Record Schema (Input CSV)

Field	Type	Required	Notes
timestamp	ISO-8601 or epoch seconds	Yes	Rows are sorted by time after parsing.
device	string	Yes	Grouping key for all per-device logic.
src_ip, dst_ip	string	Yes	Classed as LAN or external using RFC 1918, loopback, link-local and multicast ranges.
dst_port	integer	Yes	Destination port.
protocol	string	Yes	Upper-cased (TCP, UDP and so on).
bytes_out	integer	Yes	Bytes sent by the device in the flow.
bytes_in, packets, duration	number	No	Optional; accepted for compatibility.

The engine returns a structured result:

```

finding = { device, rule, severity, title, detail, mitre,
            evidence: {dst_ip, dst_port, flows, interval_s, jitter_cv, ...},
            first_seen, last_seen }
device   = { device, flows, bytes_out, src_ips[], score, status, findings }
result   = { summary: {flows, devices, findings, compromised, suspicious, with_baseline},
            devices[], findings[], quarantined[] }
  
```

4.8.3 Detection Rule Design

Table 4.5: Detection Rules, Thresholds and ATT&CK Mapping

Rule	Severity	Fires when	ATT&CK
known_bad_ip	Critical	Any flow to an address on the KNOWN_BAD_IPS threat-intelligence list.	T1071
suspicious_port	High	Destination port in SUSPICIOUS_PORTS (23, 2323, 4444, 6667, 1337, 31337, 9001, 12345).	T1571
unusual_port	Medium	Destination port outside the device baseline plus the default camera set (53, 80, 123, 443, 554, 1883, 3702, 5353, 8554, 8883); suppressed if already flagged.	T1571
new_external_dst	Medium	Baseline only: non-LAN destination never seen in the baseline.	T1071

Rule	Severity	Fires when	ATT&CK
beaconing	High	At least 6 flows to one destination:port with inter-arrival jitter (CV) ≤ 0.25 , size CV ≤ 0.5 , mean gap ≥ 1 s, and a peer outside the baseline.	T1071 / T1029
scan_hosts / scan_ports	High	At least 10 distinct destination hosts (or ports) within any 60-second window.	T1018 / T1046
exfil_volume	High	At least 5 MB sent to a single non-LAN address.	T1041
volume_outlier	Medium	Baseline only: a flow above baseline mean + 4 standard deviations and above 100 kB.	T1030

Two design decisions matter most. First, beaconing is tested only for peers outside the device's baseline, because healthy traffic is also periodic; with no baseline, regular traffic on well-known service ports is ignored unless the peer is on the known-bad list. Second, beaconing is measured by the coefficient of variation (standard deviation divided by the mean) of the gaps between flows: a 15-second timer with about 4 percent jitter scores about 0.02, far below the 0.25 limit, whereas interactive traffic scores much higher.

4.8.4 Risk Scoring

Each finding adds a weight (critical 40, high 25, medium 10, low 3) and a device's score is the sum capped at 100. A score below 10 is **clean**, 10 to 49 is **suspicious** and 50 or above is **compromised**. One critical and one high finding are therefore enough for compromised, while a single unusual port only raises suspicion.

4.8.5 Laboratory Design

Table 4.6: Laboratory Addressing Plan

Host	IP address	Networks	Published port
rtsp	10.50.0.2	lab, edge	8554 (RTSP)
cam-lobby	10.50.0.11	lab, edge	8081 (admin page)
cam-garage (and trojan sidecar)	10.50.0.12	lab only	none
c2	10.50.0.66	lab only	none
detector	10.50.0.100	lab, edge	8080 (web interface)

The lab network is declared internal, so no container on it can reach the internet. The trojan simulator shares the network namespace of cam-garage, so every connection it makes carries that camera's address, as a trojan on the real device would.

4.8.6 Security Design

- No real malware is used; the simulator reproduces indicators only and contains no payload.
- Uploads are limited to 25 MB and parsed defensively; more than 50 malformed rows aborts the parse.

- Device names used as file names must match a strict pattern, which blocks path traversal.
- All data shown in the interface is HTML-escaped.
- Sample names are checked against a fixed list before any file is opened.

4.9 Implementation

4.9.1 Core Logic

The parser maps column names to canonical names and their aliases and raises a readable error when a required column is missing:

```
cols = {c.strip().lower(): c for c in reader.fieldnames}
mapping: dict[str, str] = {}
for canon in REQUIRED + ["bytes_in", "packets", "duration"]:
    for name in [canon] + ALIASES.get(canon, []):
        if name in cols:
            mapping[canon] = cols[name]
            break
missing = [c for c in REQUIRED if c not in mapping]
if missing:
    raise FlowError(f"missing required column(s): {'', '.join(missing)}. "
                    f"Found: {'', '.join(reader.fieldnames)}")
rows, bad = [], 0
```

The beaconing rule skips peers that are part of the baseline, then tests the remaining groups for regular timing and size:

```
for (ip, port), fs in groups.items():
    if len(fs) < BEACON_MIN_EVENTS:
        continue
    # Legit services (NTP, RTSP, cloud heartbeat) are periodic too. Only treat timing as
    a
    # beacon when the peer is outside the device's baseline, or -- with no baseline --
    when
    # the port isn't a well-known service port for this device class.
    if prof is not None:
        if (ip, port) in prof["pairs"]:
            continue
    elif port in DEFAULT_ALLOWED_PORTS and ip not in KNOWN_BAD_IPS:
        continue
    ts = [f["ts"] for f in fs]
    gaps = [b - a for a, b in zip(ts, ts[1:]) if b - a > 0]
    if len(gaps) < BEACON_MIN_EVENTS - 1:
        continue
    cv, bcv = _cv(gaps), _cv([float(f["bytes_out"]) for f in fs])
    if cv <= BEACON_MAX_CV and bcv <= BEACON_MAX_BYTES_CV and statistics.fmean(gaps) >=
1:
```

Scores and statuses are derived from the findings:

```
devices = []
for dev, fl in by_dev.items():
    fs = [f for f in findings if f["device"] == dev]
    score = min(100, sum(SEVERITY_SCORE[f["severity"]] for f in fs))
    status = "compromised" if score >= 50 else "suspicious" if score >= 10 else "clean"
    devices.append({"device": dev, "flows": len(fl), "bytes_out": sum(f["bytes_out"] for
f in fl),
```

```
"status": status,
"src_ips": sorted({f["src_ip"] for f in fl}), "score": score,
"findings": len(fs))
```

4.9.2 Web Application and API

Table 4.7: HTTP Endpoints

Endpoint	Purpose
GET /	Single-page user interface.
POST /api/analyze	Multipart upload of flows and an optional baseline; returns the result JSON (400 on bad CSV, 413 above 25 MB).
POST /api/analyze/sample	Runs the bundled infected or clean sample against the bundled baseline.
POST /api/analyze/live	Analyses the live capture written by the laboratory.
POST /api/baseline/live	Freezes the current live capture as the baseline.
POST, DELETE /api/quarantine/<device>	Creates or removes the quarantine marker for a device.
GET /health	Liveness check.

```
@app.post("/api/quarantine/<device>")
def quarantine(device):
    """Response action. In the lab this drops a marker file that the trojan simulator
    honours."""
    if not re.fullmatch(r"[A-Za-z0-9_.-]{1,64}", device):
        return jsonify(error="invalid device name"), 400
    QUARANTINE.mkdir(parents=True, exist_ok=True)
    (QUARANTINE / device).write_text("quarantined by detector\n")
    return jsonify(ok=True, device=device)
```

4.9.3 Simulators

Each simulated camera serves an admin page with the deliberately weak login admin/admin and writes realistic flows: about 2 MB of video to the recorder every ten seconds, and a periodic NTP query, DNS query and cloud heartbeat. The trojan simulator drops a hidden file containing the EICAR test string and a cron.d entry, beacons to the fake C2 every 15 seconds, performs a short scan-like burst on port 23, logs one large-upload record and, when a quarantine marker appears, deletes its files and exits:

```
def beacon():
    t0 = time.time()
    out = random.randint(290, 330)
    try:
        with socket.create_connection((C2, C2_PORT), timeout=3) as s:
            s.sendall(f"id={DEVICE}&seq={int(t0)}".encode().ljust(out, b"."))
            s.recv(16)
    except OSError as e:
        print(f"[sim] beacon failed: {e}", flush=True)
    flowlog.log(DEVICE, IP, C2, C2_PORT, "TCP", out, 3, time.time() - t0)
```

4.9.4 User Interface

The interface is a single HTML page. Figures 4.2 to 4.5 show its main states; the red numbered boxes match the numbered explanations beneath each figure.

Figure 4.2: Upload screen (landing page)

- (1) Capture to analyse (required). The flow CSV to inspect: one row per network flow, exported from Zeek, Suricata, a router or the lab simulators.
- (2) Known-good baseline (optional). A CSV of normal traffic. It teaches the engine each device's usual ports and destinations; without it only built-in rules apply.
- (3) Analyse. Uploads both files to POST /api/analyze and renders the result below the card.
- (4) Schema hint. The seven required columns are generated from engine.REQUIRED; common aliases and ISO or epoch timestamps are also accepted.
- (5) Load sample: infected / clean. Runs a bundled capture against the bundled baseline, so you can try the tool with no files.
- (6) Analyse live lab capture. Reads flows.csv written by the Docker lab's simulated cameras and trojan.
- (7) Freeze live capture as baseline. Declares the traffic seen so far as “known good”; used in step 2 of the demo, before the trojan starts.

Device	Status	Risk	Flows	Bytes out	Findings
cam-garage 10.50.0.12	compromised	100	560	730.8 MB	22
cam-lobby 10.50.0.11	clean	0	397	724.9 MB	0
cam-office 10.50.0.13	clean	0	396	719.2 MB	0

Figure 4.3: Results: summary tiles and device table

- (1) Summary tiles. Flows analysed, devices seen, total findings, how many devices are compromised or suspicious, and whether a baseline was used.
- (2) Status pill. clean (score < 10), suspicious (10–49) or compromised (50+). Rows are sorted riskiest first.

- (3) Risk score (0–100): the sum of finding weights (critical 40, high 25, medium 10, low 3), capped at 100.
- (4) Bytes out. Total data the device sent; a quick sanity check next to the exfiltration findings.
- (5) Quarantine. Response action, shown only for devices that are not clean.

Findings					
Severity	Device	Finding	Detail	ATT&CK	Window (UTC)
critical	cam-garage	Contact with known-bad IP	121 flow(s) to threat-intel listed address 203.0.113.66	T1071 Application Layer Protocol (C2)	2026-01-01 11:10:00 2026-01-01 11:39:55
critical	cam-garage	Contact with known-bad IP	1 flow(s) to threat-intel listed address 198.51.100.77	T1071 Application Layer Protocol (C2)	2026-01-01 11:31:00 2026-01-01 11:31:00
high	cam-garage	Suspicious port 4444 (Metasploit default)	121 flow(s) to port 4444; destinations: 203.0.113.66	T1571 Non-Standard Port	2026-01-01 11:10:00 2026-01-01 11:39:55
high	cam-garage	Suspicious port 23 (Telnet)	30 flow(s) to port 23; destinations: 10.50.0.100, 10.50.0.101, 10.50.0.102, 10.50.0.103, 10.50.0.104	T1571 Non-Standard Port	2026-01-01 11:19:00 2026-01-01 11:19:23
high	cam-garage	Suspicious port 2323 (Telnet-alt)	1 flow(s) to port 2323; destinations: 10.50.0.20	T1571 Non-Standard Port	2026-01-01 11:19:31 2026-01-01 11:19:31
high	cam-garage	Periodic beaconing to 203.0.113.66:4444	121 flows, mean interval 15.0s (jitter CV 0.02), avg 310 B/flow	T1071 / T1029 Scheduled Transfer	2026-01-01 11:10:00 2026-01-01 11:39:55
high	cam-garage	Scan-like burst across 10 distinct hosts	10 distinct hosts within 60s window	T1018 Remote System Discovery	2026-01-01 11:18:11 2026-01-01 11:19:05
... 15 more findings not shown					

Figure 4.4: Findings table

- (1) Severity. Critical and high findings come first. Colour and text both carry the severity, so it is readable without colour.
- (2) Finding. What was detected, e.g. contact with a known-bad IP or periodic beaconing.
- (3) Detail. The evidence behind it: flow counts, mean beacon interval, jitter, megabytes sent.
- (4) ATT&CK. The MITRE technique the behaviour maps to, handy for SOC reports.
- (5) Window (UTC). First and last time the behaviour was seen in the capture.

Devices					
Device	Status	Risk	Flows	Bytes out	Findings
cam-garage 10.50.0.12	compromised	100	560	730.8 MB	22
cam-lobby 10.50.0.11	clean	0	397	724.9 MB	0
cam-office 10.50.0.13	clean	0	396	719.2 MB	0

Figure 4.5: Quarantine and release

- (1) The device is still reported as compromised: quarantine is a response action, not a verdict change.
- (2) Release replaces the Quarantine button and calls DELETE /api/quarantine/<device>, which removes the marker.

The layout is responsive. On a phone the cards stack, the summary tiles reflow into two columns and the tables scroll inside their own cards, so the page itself never scrolls sideways.

1353 Flows		3 Devices	
22 Findings		1 Compromised	
0 Suspicious		yes Baseline	

Devices					
Device	Status	Risk	Flows	By ou	
cam-garage 10.50.0.12	compromised	100	560	73 Mb	
cam-lobby 10.50.0.11	clean	0	397	72 Mb	
cam-office 10.50.0.13	clean	0	396	71 Mb	

Figure 4.6: Mobile: summary and devices

- (1) Tiles reflow to two columns.
- (2) Device name and source IP are in the first column; swipe the table sideways to reach status, risk, bytes out and the Quarantine action. The page itself does not scroll sideways.

4.10 Testing and Results

4.10.1 Automated Tests

The project has 15 pytest tests that run in under a second without Docker; all 15 passed at the time of writing.

Table 4.8: Automated Tests and What They Prove

Test	Type	Behaviour verified
test_upload_with_baseline	Positive	Analysing the infected sample with a baseline upload gives one compromised device and with_baseline = true.
test_upload_rejects_bad_csv	Negative	A CSV missing required columns returns HTTP 400 with a clear message.
test_no_file	Negative	A request with no file returns HTTP 400.
test_sample_and_quarantine_roundtrip	Both	Sample analysis finds one compromised device; an invalid sample name is rejected; quarantine and release create and remove the marker.
test_quarantine_rejects_traversal	Negative	A path-traversal device name is refused.
test_clean_capture_has_no_findings (x2)	Negative	The clean capture gives no findings, with and without a baseline.
test_infected_device_flagged_others_clean (x2)	Positive	Only cam-garage is compromised and the expected rules fire; the other cameras stay clean.
test_beacon_interval_reported	Positive	The beacon destination is 203.0.113.66 and the interval is between 14 and 16 seconds.
test_column_aliases_and_epoch	Positive	A CSV with aliases and epoch timestamps parses and a beacon is detected.
test_missing_columns_message	Negative	The error message names the missing column.
test_empty_or_headers_only (x2)	Negative	An empty file and a headers-only file are rejected.
test_irregular_traffic_not_beacon	Negative	Thirty flows with random gaps and sizes are not reported as beaconing.

4.10.2 Defects Found and Their Remediation

Table 4.9: Defects, Remediation and Status

#	Severity	Defect	Remediation / status
1	High	The first beaconing rule reported legitimate periodic traffic (video, NTP, cloud heartbeats) as beacons, so the clean capture produced findings.	Beaconing now applies only to peers outside the baseline (or, with no baseline, off well-known service ports). Fixed.
2	High	The large-transfer rule did not fire because Python's ipaddress.is_private also covers the documentation ranges 198.51.100.0/24 and 203.0.113.0/24 used for the lab's stand-in internet hosts.	LAN membership is decided by an explicit list of private, loopback, link-local and multicast ranges. Fixed.

#	Severity	Defect	Remediation / status
3	Medium	The baseline is held in process memory, so it is lost on restart and may not be shared between server workers.	Documented as a limitation; single worker or shared storage recommended. Open.

4.10.3 Detection Results on the Sample Captures

The infected sample (1,353 flows across 3 cameras) was analysed against the baseline. The engine reported 22 findings (2 critical, 7 high, 13 medium), all on cam-garage, which scored 100 and was marked compromised. Table 4.10 shows one finding for each rule that fired and Table 4.11 compares the per-device outcome for the infected and clean captures.

Table 4.10: One Finding per Rule from the Infected Sample

Severity	Finding	Evidence	ATT&CK
Critical	Contact with known-bad IP	121 flow(s) to threat-intel listed address 203.0.113.66	T1071 Application Layer Protocol (C2)
High	Suspicious port 4444 (Metasploit default)	121 flow(s) to port 4444; destinations: 203.0.113.66	T1571 Non-Standard Port
High	Periodic beaconing to 203.0.113.66:4444	121 flows, mean interval 15.0s (jitter CV 0.02), avg 310 B/flow	T1071 / T1029 Scheduled Transfer
High	Scan-like burst across 10 distinct hosts	10 distinct hosts within 60s window	T1018 Remote System Discovery
High	Scan-like burst across 10 distinct ports	10 distinct ports within 60s window	T1046 Network Service Discovery
High	Large outbound transfer to 198.51.100.77	8.5 MB sent to external address	T1041 Exfiltration Over C2 Channel
Medium	Unusual destination port 22	1 flow(s) on port 22 outside the device's baseline profile	T1571 Non-Standard Port
Medium	New external destination 203.0.113.66	121 flow(s) to an address never seen in the baseline	T1071 Application Layer Protocol
Medium	Flow size far above baseline	1 flow(s) exceed baseline mean+4 σ (4,302,583 B); max 8,500,000 B	T1030 Data Transfer Size Limits

Table 4.11: Per-Device Results, Infected and Clean Captures

Capture	Device	Flows	Risk score	Status	Findings
Infected	cam-garage	560	100	compromised	22
Infected	cam-lobby	397	0	clean	0
Infected	cam-office	396	0	clean	0
Clean	cam-lobby	397	0	clean	0
Clean	cam-garage	396	0	clean	0
Clean	cam-office	397	0	clean	0

4.10.4 Positive Functional Tests**Table 4.12: Positive Test Cases**

#	Test case	Expected result	Outcome
P1	Analyse the infected sample with the baseline.	cam-garage compromised; two other cameras clean.	Pass
P2	Analyse the clean sample with and without the baseline.	Zero findings; all devices clean.	Pass
P3	Upload the infected capture and baseline through the API.	HTTP 200; one compromised device.	Pass
P4	Upload a CSV using aliases and epoch timestamps.	Parsed correctly; beacon detected.	Pass
P5	Check the reported beacon.	Destination 203.0.113.66; interval about 15 s.	Pass
P6	Quarantine cam-garage, then release it.	Marker created and listed; marker removed on release.	Pass
P7	Run the simulators locally and analyse the live capture.	Flows detected; quarantine stops the simulator.	Pass

4.10.5 Negative Tests**Table 4.13: Negative Test Cases**

#	Test case	Expected result	Outcome
N1	Upload a CSV missing required columns.	HTTP 400 naming the missing columns.	Pass
N2	Upload an empty or headers-only file.	Rejected with an error message.	Pass
N3	Call the analyse endpoint with no file.	HTTP 400.	Pass
N4	Request a sample named ../etc/passwd.	HTTP 400; no file access.	Pass
N5	Quarantine a device name containing a path-traversal sequence.	Request refused.	Pass
N6	Analyse thirty flows with irregular gaps and sizes.	No beaconing finding.	Pass

#	Test case	Expected result	Outcome
N7	Analyse the clean capture.	No findings (no false positives).	Pass

4.10.6 Laboratory Run of the Simulators

No Docker daemon was available in my development environment, so I exercised the laboratory logic by running the simulators directly as Python processes. The fake C2 server logged the beacons; the shared CSV held 21 flow records; and the engine, run without a baseline, reported five findings on the device (a known-bad address, suspicious port 23, a scan burst, a large transfer to 198.51.100.77 and an unusual port) and marked it compromised. When I created the quarantine marker, the simulator reported it, removed its two persistence files and exited. In this accelerated one-second run no beaconing finding appeared; the 15-second cadence of the real simulator is covered by the sample-data tests. The container layer itself (networking, RTSP, volume sharing) has still to be validated with docker compose up --build on a machine with Docker.

4.11 Discussion of Results

The results show that useful detection is possible from flow data alone. No single rule is proof of compromise, since an unusual port may be a mistake and a periodic flow may be a heartbeat, but the combination is persuasive: in the infected sample the same camera contacts a known-bad address, beacons at a fixed interval to a peer it never used before, scans the network and sends a large volume to an unknown host, and its score reflects that accumulation. The clean capture, which is just as regular, produces no findings, which shows that the baseline logic does its job.

The most instructive result was the false-positive defect. A detector that treats all regularity as suspicious is unusable on cameras, and the fix, judging regularity only against peers outside the baseline, is a small change that makes the difference between a toy and a tool.

4.12 Challenges Encountered

The main challenges are listed in Table 3.1. In addition, choosing thresholds without real attack data was difficult: the values in the engine (six events, a coefficient of variation of 0.25, ten distinct targets in 60 seconds, 5 MB for exfiltration) are reasonable starting points but need tuning against real traffic.

4.13 Limitations

- The rules are heuristic and were tested on simulated data; a patient attacker could vary intervals and sizes beyond the thresholds.
- Encrypted payloads cannot be inspected; the system reasons only about metadata.
- Laboratory telemetry is generated by the simulators, not captured from a wire.

- Quarantine in the laboratory is a marker file; the application has no built-in authentication.
- The baseline is held in memory and may not be shared between server workers.
- The Docker container layer has not been run end to end in my environment.

4.14 Recommendations and Future Work

- Ingest real telemetry from Zeek, Suricata or NetFlow/IPFIX and map addresses to device names from DHCP or an asset inventory.
- Store baselines in a database or file and refresh them on a schedule.
- Place the application behind a TLS reverse proxy with single sign-on, and connect quarantine to a firewall or network access control system.
- Load threat-intelligence feeds into the known-bad list and tune thresholds on real traffic.
- Evaluate on public IoT datasets and measure detection and false-positive rates; consider learned baselines and streaming analysis.

4.15 Conclusion

The mini project achieved its objectives. I designed, implemented and tested an agentless, flow-based Trojan Horse detection system for IoT devices, together with a safe Docker laboratory in which infection, detection and containment can be demonstrated repeatably. All 15 automated tests pass, the clean capture produces no false positives, and the infected capture is identified with one compromised device and no collateral flags.

CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This report has described my SIWES training at HiiT Plc. Chapter One explained the origin, aims and structure of SIWES. Chapter Two presented the history, services and units of HiiT Plc, my placement details, the work I completed and a review of the organisation. Chapter Three recorded the experience I gained in network analysis, cybersecurity, containers, Python development, testing, version control and documentation. Chapter Four presented the mini project: the design, implementation and testing of a network-flow-based Trojan Horse detection system for IoT devices and its simulated laboratory.

5.2 Knowledge and Skills Acquired

- Reading network-flow data for behaviour and distinguishing normal regularity from malicious regularity.
- Describing threats with MITRE ATT&CK and studying malicious behaviour safely and ethically.
- Building isolated, repeatable laboratories with Docker Compose.
- Designing layered, testable Python programs with a Flask web interface and JSON API.
- Writing automated tests, diagnosing defects and recording their remediation.
- Using Git and GitHub for version control and review.
- Producing professional documentation, including automated screenshots, a PDF guide and a Word report.
- Time management, communication and ethical conduct in a technical workplace.

5.3 Conclusion

SIWES did what it is meant to do. It turned the concepts from my courses into skills I can demonstrate, exposed me to the standards of a professional organisation and sharpened my interest in cybersecurity. I leave the placement better prepared for the remainder of my degree and for a career in software engineering and security.

5.4 Recommendations

5.4.1 To the Host Organisation

- Give interns a written task schedule and learning outcomes at the start of the placement.
- Assign each intern a project with a named reviewer and a defined deliverable.
- Provide backup power and redundant connectivity for laboratories.
- Arrange supervised portfolio presentations at the end of the placement.

5.4.2 To the University and the Department

- Introduce practical security, containers and version-control content earlier in the programme.
- Increase the number and quality of supervisory visits and feedback during the placement.
- Keep a record of host organisations rated by past students.

5.4.3 To the ITF and Regulators

- Pay student allowances promptly.
- Strengthen monitoring so that tasks match the student's discipline.
- Encourage more technology companies to host ICT students.

5.4.4 To Future SIWES Students

- Treat the logbook as a daily habit and not a last-week task.
- Build a portfolio of documented projects during the placement.
- Ask questions, accept correction and keep ethical boundaries in every technical activity.

REFERENCES

- Antonakakis, M., April, T., Bailey, M., et al. (2017). Understanding the Mirai botnet. Proceedings of the 26th USENIX Security Symposium, Vancouver.
- BusinessDay. (2020). HiiT Plc offers N20 million digital skills acquisition scholarship to Nigerian students. <https://businessday.ng/technology/article/hiit-plc-offers-n20-million-digital-skills-acquisition-scholarship-to-nigerian-students/>
- Claise, B. (Ed.). (2004). RFC 3954: Cisco Systems NetFlow Services Export Version 9. Internet Engineering Task Force.
- Claise, B., Trammell, B., & Aitken, P. (Eds.). (2013). RFC 7011: Specification of the IP Flow Information Export (IPFIX) Protocol. Internet Engineering Task Force.
- Docker, Inc. (n.d.). Docker and Docker Compose documentation. <https://docs.docker.com>
- EICAR. (n.d.). Anti-malware testing standard file. <https://www.eicar.org>
- Federal Republic of Nigeria. (2015). Cybercrimes (Prohibition, Prevention, etc.) Act, 2015.
- HiiT Plc. (n.d.). Company profile and training programme information. Lagos, Nigeria. **[add the page or document you used]**
- Industrial Training Fund. (n.d.). Students' Industrial Work Experience Scheme (SIWES): information and guidelines. Jos, Nigeria: ITF.
- MITRE Corporation. (n.d.). MITRE ATT&CK. <https://attack.mitre.org>
- Paxson, V. (1999). Bro: a system for detecting network intruders in real time. Computer Networks, 31(23-24).
- Pallets Projects. (n.d.). Flask documentation. <https://flask.palletsprojects.com>

APPENDIX A: CORE SOURCE CODE OF THE DETECTION SYSTEM

The complete source is in the project repository (trojan-lab/). The excerpts below are the parts discussed in Chapter Four.

A.1 Address classification (engine.py)

```
_INTERNAL = [ipaddress.ip_network(n) for n in (
    "10.0.0.0/8", "172.16.0.0/12", "192.168.0.0/16", "127.0.0.0/8", "169.254.0.0/16",
    "224.0.0.0/4", "fe80::/10")]

def _is_private(ip: str) -> bool:
    """LAN-side address. Deliberately NOT ipaddress.is_private: that also covers the
    TEST-NET documentation ranges, which the lab uses to stand in for internet hosts."""
    try:
        a = ipaddress.ip_address(ip)
    except ValueError:
        return False
    return any(a in n for n in _INTERNAL if n.version == a.version)
```

A.2 Baseline construction (engine.py)

```
def build_baseline(flows: list[dict]) -> dict:
    """Per-device profile of normal behaviour from known-good traffic."""
    prof: dict[str, dict] = defaultdict(lambda: {"ports": set(), "dsts": set(), "pairs":
    set(), "bytes": []})
    for f in flows:
        p = prof[f["device"]]
        p["ports"].add(f["dst_port"])
        p["dsts"].add(f["dst_ip"])
        p["pairs"].add((f["dst_ip"], f["dst_port"]))
        p["bytes"].append(f["bytes_out"])
    out = {}
    for dev, p in prof.items():
        b = p["bytes"]
        out[dev] = {"ports": p["ports"], "dsts": p["dsts"], "pairs": p["pairs"],
        "flows": len(b),
                    "mean_bytes": statistics.fmean(b),
                    "std_bytes": statistics.pstdev(b) if len(b) > 1 else 0.0}
    return out
```

A.3 Beaconing rule (engine.py)

```
for (ip, port), fs in groups.items():
    if len(fs) < BEACON_MIN_EVENTS:
        continue
    # Legit services (NTP, RTSP, cloud heartbeat) are periodic too. Only treat timing as
    a
    # beacon when the peer is outside the device's baseline, or -- with no baseline --
    when
    # the port isn't a well-known service port for this device class.
    if prof is not None:
        if (ip, port) in prof["pairs"]:
            continue
    elif port in DEFAULT_ALLOWED_PORTS and ip not in KNOWN_BAD_IPS:
        continue
    ts = [f["ts"] for f in fs]
    gaps = [b - a for a, b in zip(ts, ts[1:])] if b - a > 0
    if len(gaps) < BEACON_MIN_EVENTS - 1:
        continue
```

```

cv, bcv = _cv(gaps), _cv([float(f["bytes_out"]) for f in fs])
if cv <= BEACON_MAX_CV and bcv <= BEACON_MAX_BYTES_CV and statistics.fmean(gaps) >=
1:
    sev = "high"
    findings.append(_finding(dev, "beaconing", sev, f"Periodic beaconing to {ip}:
{port}",
        f"{len(fs)} flows, mean interval {statistics.fmean(gaps):.1f}s (jitter CV
{cv:.2f})",
        f"avg {statistics.fmean(f['bytes_out'] for f in fs):.0f} B/flow",
        "T1071 / T1029 Scheduled Transfer",
        {"dst_ip": ip, "dst_port": port, "flows": len(fs), "interval_s":
round(statistics.fmean(gaps), 2),
        "jitter_cv": round(cv, 3)}, ts[0], ts[-1]))

# 5. Scan burst: many distinct targets inside a short window
for key, label in (("dst_ip", "hosts"), ("dst_port", "ports")):
    lo, hit = 0, None

```

A.4 Scan-burst rule (engine.py)

```

# 5. Scan burst: many distinct targets inside a short window
for key, label in (("dst_ip", "hosts"), ("dst_port", "ports")):
    lo, hit = 0, None
    for hi in range(len(fl)):
        while fl[hi]["ts"] - fl[lo]["ts"] > SCAN_WINDOW_S:
            lo += 1
        win = fl[lo:hi + 1]
        if len({x[key] for x in win}) >= SCAN_MIN_TARGETS:
            hit = (win[0]["ts"], win[-1]["ts"], len({x[key] for x in win}))
            break
    if hit:
        findings.append(_finding(dev, f"scan_{label}", "high", f"Scan-like burst across
{hit[2]} distinct {label}",
            f"{hit[2]} distinct {label} within {SCAN_WINDOW_S}s window",
            "T1046 Network Service Discovery" if label == "ports" else "T1018 Remote
System Discovery",
            {"distinct": hit[2], "window_s": SCAN_WINDOW_S, hit[0], hit[1]})

```

A.5 Quarantine endpoints (app.py)

```

@app.post("/api/quarantine/<device>")
def quarantine(device):
    """Response action. In the lab this drops a marker file that the trojan simulator
honours."""
    if not re.fullmatch(r"[A-Za-z0-9_-]{1,64}", device):
        return jsonify(error="invalid device name"), 400
    QUARANTINE.mkdir(parents=True, exist_ok=True)
    (QUARANTINE / device).write_text("quarantined by detector\n")
    return jsonify(ok=True, device=device)

@app.delete("/api/quarantine/<device>")
def release(device):
    if re.fullmatch(r"[A-Za-z0-9_-]{1,64}", device):
        (QUARANTINE / device).unlink(missing_ok=True)
    return jsonify(ok=True)

```

A.6 Trojan simulator main loop (kworker_upd.py)

```

def main():

```

```

delay = int(os.environ.get("START_DELAY", "0"))
print(f"[sim] starting in {delay}s", flush=True)
time.sleep(delay)
install()
n = 0
while True:
    if os.path.exists(QUARANTINE):
        cleanup()
        print("[sim] quarantine detected: persistence removed, process exiting",
flush=True)
        return
    beacon()
    n += 1
    if n == 4:
        scan_burst()
    if n == 8: # a log record only -- no data is transmitted
        flowlog.log(DEVICE, IP, "198.51.100.77", 443, "TCP", 8_500_000, 400, 20)
    time.sleep(INTERVAL * (1 + random.uniform(-0.04, 0.04)))

```

A.7 Laboratory services (docker-compose.yml)

```

trojan:
  profiles: ["trojan"]
  build: { context: ., dockerfile: trojan_sim/Dockerfile }
  network_mode: "service:cam-garage"
  environment: { DEVICE: cam-garage, C2_HOST: 10.50.0.66, START_DELAY: "5" }
  volumes: ["labdata:/data"]
  depends_on: [cam-garage, c2]

detector:

```

APPENDIX B: SIWES LOGBOOK EXTRACT

[Attach scanned or typed pages of the SIWES logbook signed by the industry-based supervisor. The table below can be used as a weekly summary.]

Table B.1: Weekly Summary of Activities

Week	Dates	Activities carried out	Supervisor's remark
Week 1	[dates]	[activities]	[remark]
Week 2	[dates]	[activities]	[remark]
Week 3	[dates]	[activities]	[remark]
Week 4	[dates]	[activities]	[remark]
Week 5	[dates]	[activities]	[remark]
Week 6	[dates]	[activities]	[remark]
Week 7	[dates]	[activities]	[remark]
Week 8	[dates]	[activities]	[remark]
Week 9	[dates]	[activities]	[remark]
Week 10	[dates]	[activities]	[remark]
Week 11	[dates]	[activities]	[remark]
Week 12	[dates]	[activities]	[remark]

APPENDIX C: CERTIFICATES AND SUPPORTING DOCUMENTS

[Attach the SIWES completion letter, training certificate(s), and any other supporting documents.]

APPENDIX D: GLOSSARY OF TECHNICAL ACRONYMS

Acronym	Definition
ATT&CK	Adversarial Tactics, Techniques and Common Knowledge (MITRE)
C2	Command and Control
CPN	Computer Professionals Registration Council of Nigeria
CV	Coefficient of Variation (standard deviation divided by mean)
EICAR	European Institute for Computer Antivirus Research (test file standard)
IoT	Internet of Things
IPFIX	IP Flow Information Export
ITF	Industrial Training Fund
NTP	Network Time Protocol
RTSP	Real Time Streaming Protocol
SIWES	Student Industrial Work Experience Scheme
SWOT	Strengths, Weaknesses, Opportunities, Threats
TLS	Transport Layer Security